

Projeto Eventua

Gustavo Brunoni

1. Qual foi sua principal contribuição no trabalho?

Minha principal contribuição foi o desenvolvimento de toda a camada de apresentação (views EJS), a estilização visual da plataforma (CSS) e a lógica de negócio dos dois módulos centrais: eventos e inscrições. Fui responsável por implementar a regra de controle de vagas e o ciclo completo de inscrição e cancelamento, que é o coração funcional da plataforma.

2. Quais partes do projeto você desenvolveu?

Desenvolvi os seguintes arquivos e funcionalidades:

- **eventoService.js** — criação, edição e remoção de eventos com validação e recalculo de vagas.
- **inscricaoService.js** — lógica de inscrição (checagem de vagas, reativação de cancelada) e cancelamento com devolução de vaga.
- **eventoController.js** — home, listagem com filtro por tipo, detalhe do evento e dashboard do participante.
- **inscricaoController.js** — ações de inscrever e cancelar com tratamento de erros.
- **14 views EJS** — partials, autenticação, eventos, dashboards e páginas de erro (403, 404).
- **public/css/style.css** — toda a estilização da interface.
- **public/js/main.js** — scripts client-side.

3. Qual foi o trecho mais complicado do código? Explique o motivo.

O trecho mais complexo foi a função inscrever no `inscricaoService.js`, porque envolve múltiplas verificações encadeadas e duas operações de escrita que precisam ser consistentes:

```
// 1. verifica se evento existe e esta ativo
// 2. verifica se usuario ja esta inscrito
// 3. verifica se ainda ha vagas
eventoRepository.atualizar(eventoId, { vagas: novasVagas }); // decrementa
inscricaoRepository.criar(novaInscricao);                    // registra
```

A dificuldade foi tratar a reinscrição: se o usuário já havia cancelado, o sistema não deve criar um registro duplicado — deve reativar a inscrição existente. Isso exigiu uma checagem extra com `inscricaoRepository.buscar()` e uma lógica bifurcada entre atualizar e criar. Usei a IA para prototipar essa lógica antes de implementá-la, o que me ajudou a visualizar os casos de borda e evitar bugs de duplicidade desde o início.

4. Qual foi sua maior dificuldade técnica e como resolveu?

A maior dificuldade foi manter a contagem de vagas consistente quando o admin edita a capacidade de um evento que já tem inscritos. O campo vagas não pode ser sobrescrito diretamente — é preciso recalcular levando em conta quantos já estão inscritos:

```
const inscritos = evento.capacidade - evento.vagas;
novosDados.vagas = capacidade - inscritos;
```

Resolvi isso no `eventoService.atualizar`, calculando o delta antes de salvar. Consultei a IA para confirmar se essa abordagem cobria todos os casos e para entender como o EJS lida com variáveis não definidas nos templates, usando o operador `||` para evitar erros em campos opcionais.

5. Você utilizou IA? Se sim, explique exatamente como.

Sim, utilizei IA (ChatGPT e GitHub Copilot) em três frentes:

- Prototipação e estruturação do projeto: antes de codificar, usei a IA para esboçar a estrutura das views EJS e a organização dos arquivos de estilo, o que acelerou a fase de planejamento.
- Lógica de inscrição: usei a IA para prototipar a função inscrever e identificar os casos de borda (reinscrição, vaga zero, evento inativo) antes de implementá-los.
- Trechos repetitivos nas views: o Copilot agilizou a escrita dos loops `forEach` nas listas de eventos e inscrições, onde o padrão era sempre o mesmo.

Para a lógica de negócio mais crítica (controle de vagas), não usei geração automática de código — escrevi manualmente para entender exatamente cada linha e conseguir depurar os erros.

6. O que você aprendeu de novo com este trabalho?

Aprendi como funciona o EJS na prática: como passar dados do controller para a view, usar `<% %>` para lógica e `<%= %>` para renderizar valores, e como os `partials` evitam repetição de HTML. No back-end, entendi a diferença entre `service` (regras de negócio) e `controller` (recebe a requisição e delega). Isso ficou claro quando precisei mover a lógica de vagas do controller para o service — o código ficou muito mais organizado.

7. Se tivesse mais tempo, o que melhoraria no projeto?

Adicionaria notificações por e-mail (Nodemailer) quando o usuário se inscreve ou cancela. Implementaria paginação na listagem de eventos, que hoje carrega todos os registros de uma vez. No front-end, migraria o CSS puro para Tailwind CSS com componentes reutilizáveis. Por fim, adicionaria campo de busca por nome de evento na página de listagem.